

Project Objective: Local File Manager CLI

Project Overview

The goal of this project is to create a lightweight, text-based **Command Line Interface (CLI) File Manager** using Python. This utility automates basic file system operations, allowing users to interact with their local directories directly from the terminal without needing a graphical user interface.

Core Objectives

- **Automate File Discoverability:** Automatically scan and display a recursive list of all existing files and folders within the working directory so the user always knows what files are available.
- **Provide Full CRUD Operations:** Implement the four fundamental functions of persistent storage:
 - **Create:** Prompt the user for a new filename and initial content, safely creating a new file only if it doesn't already exist.
 - **Read:** Let users select an existing file and instantly view its text content directly in the terminal.
 - **Update:** Give users a sub-menu of modifications, including **renaming** the file, **overwriting** its existing content entirely, or **appending** new information to the end of it.
 - **Delete:** Safely purge an unwanted file from the local file system based on user confirmation.
- **Implement Error Handling and Safety:** Prevent crashes by wrapping file operations in safety checks. The application must verify whether a file actually exists or if it is a directory before trying to read, modify, or delete it, and elegantly catch unexpected system errors.
- **Design a User-Friendly Menu:** Build an interactive, numeric-based routing menu that captures user inputs and executes the corresponding file operations seamlessly.

Key Concepts to Master

To build this project successfully, a developer will gain practical experience with:

1. **Object-Oriented File Paths:** Moving away from raw strings to leverage modern library tools for path comparisons and existence checks.
2. **Context Managers:** Using structured file opening and closing techniques (**with** blocks) to guarantee that system resources are never leaked.
3. **File Modes:** Understanding the practical differences between Write ('w'), Read ('r'), and Append ('a') modes.
4. **Control Flow & Exception Handling:** Managing user choices using conditional logic while protecting the runtime environment using try-except blocks.